Worksheet 4 — Injection & Input Handling (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 4 Aligned:** OWASP 2025 **A05 Injection** · **CWE-89** (SQLi), **CWE-78** (OS command injection), **CWE-434** (unrestricted upload) **Signature game:** 🐍 **SQLi Boss Fight** — each successful injection lands a "hit" on the boss; the boss falls when you dump every credential and land an RCE.

> ⚠️ **Ethics note:** All payloads here are for the provided sandbox (`vulnerable_app.py`) and your own DVWA/Juice Shop containers **only**. Never test systems you do not own or have written permission to test. Unauthorized injection is a crime under most computer-misuse laws.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	9/5/2026	

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Why does a **parameterized query** (`execute(sql, (params,))`) defeat SQL injection, while string formatting (`"... '%s'" % user`) does not? Reference how the database treats data vs. code.

Answer: Parameterized queries send the SQL structure to the database first, then pass user input as pure data — it can never turn into SQL commands. String formatting builds the final SQL text before sending it, so a quote (') in the input can break out and become part of the command itself.

2. In the `/ping` endpoint, `subprocess.run("ping -c 1 " + host, shell=True)` is vulnerable. Explain how `shell=True` turns user input into **CWE-78**, and how an argument array (`["ping","-c","1",host]`) removes the shell.

Answer: With shell=True, the whole string (including user input) is handed to the shell, so characters like ; or $() let the attacker run extra commands. An argument array (["ping","-c","1",host]) with no shell skips that interpretation — the input is passed as one plain value, not as code.

3. Distinguish **input validation** (allow-list) from **output handling**. Why is validation alone insufficient defense for SQLi?

Answer: Validation checks if input looks safe before use (allow-list). Output handling controls how input is used at the database/shell level (e.g., parameterized queries). Validation alone isn't enough because you can't block every dangerous pattern, and if the code still concatenates strings into SQL, it's still exploitable.

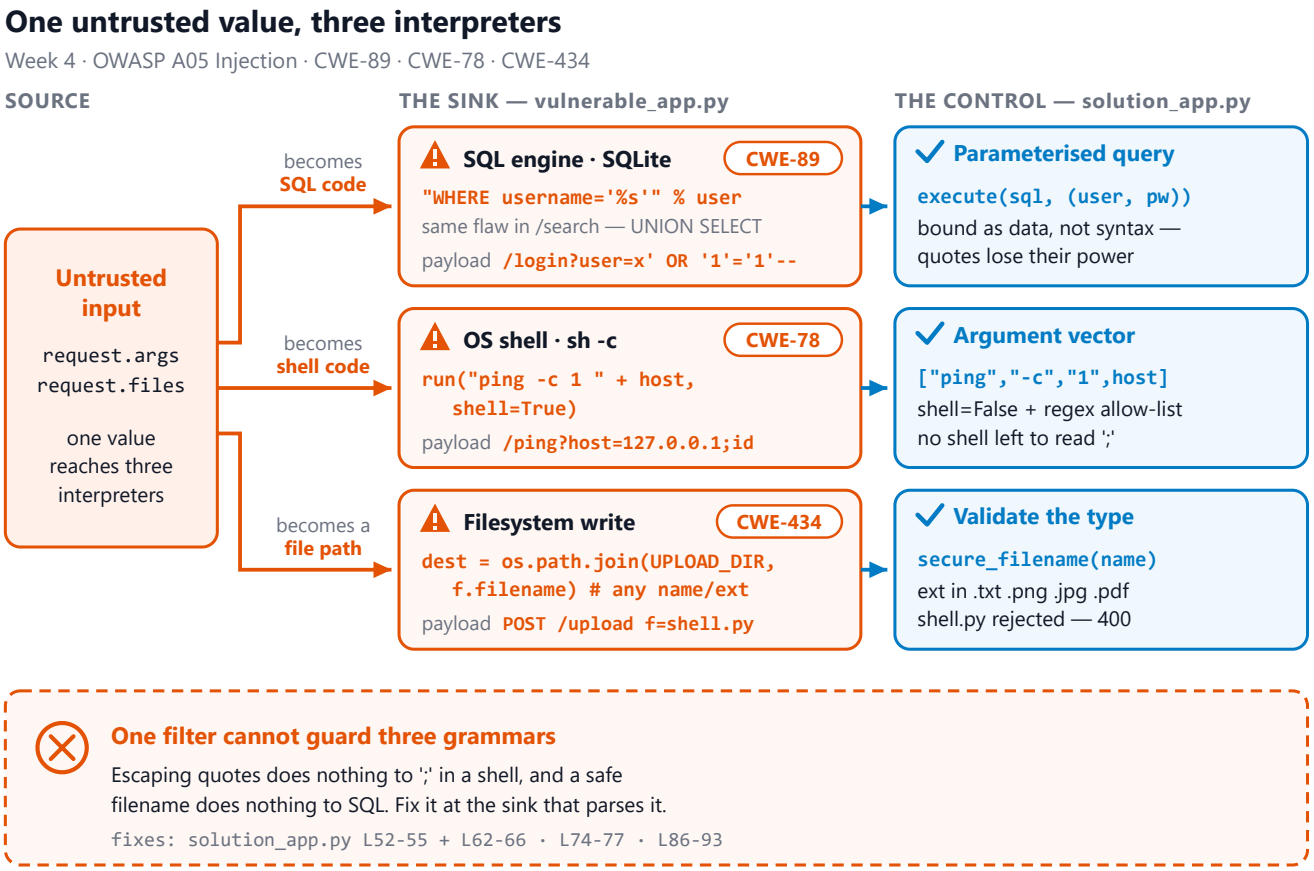
4. The `/upload` route saves any filename to disk (**CWE-434**). What two properties must a directory and a filename have for an upload to become remote code execution, and which does `solution_app.py` remove?

Answer: An upload becomes RCE if (1) the save folder is one the server will run files from, and (2) the file extension is executable (like .py) with an unsanitized filename. solution_app.py fixes #2 with an extension

allow-list and secure_filename(), but doesn't need to fix #1 since the folder was never served anyway.

5. What is a **UNION-based** SQLi, and why must the injected `SELECT` return the same number of columns as the original query? Relate to `/search?q=' UNION SELECT username,password FROM users--`.

Answer: UNION SQLi appends a second SELECT to steal extra data, merging it into the app's normal output. It must match the original query's column count because UNION joins results column-by-column — mismatched counts cause an error. Here the original query returns 2 columns, so SELECT username, password (also 2 columns) fits perfectly.



Part 3 — Hands-on Lab (150 min)

Learning goals: extract data via SQLi, achieve OS command injection, exploit an unrestricted upload, then prove each fix in `solution_app.py` blocks the payload.

Prerequisites: Docker + Docker Compose, `curl`, a browser. Working dir: `labs/week04-injection/`.

Environment setup

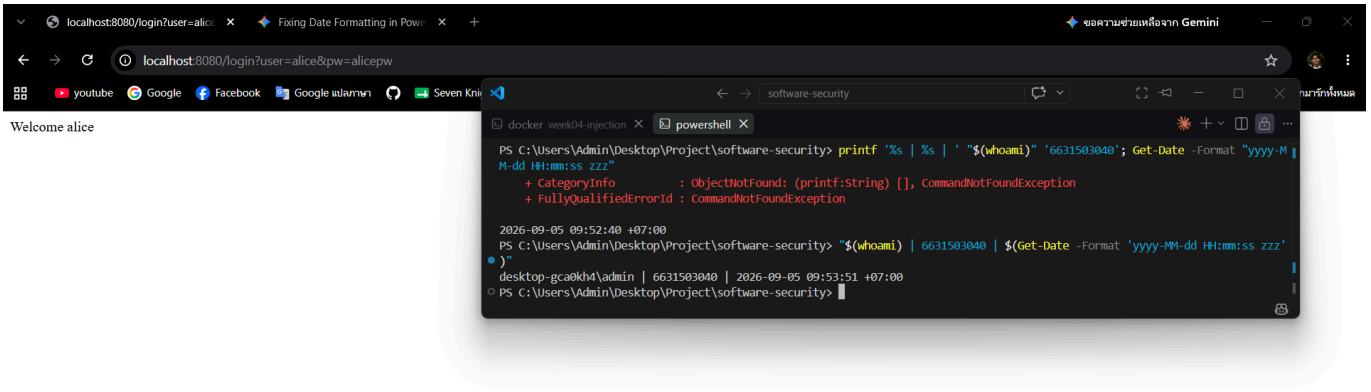
```
cd labs/week04-injection
docker compose up            # builds python:3.12-slim, installs flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: injection-lab, port 8080)
```

Optional secondary targets:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa        # DVWA  ->
http://localhost
docker run --rm -p 3000:3000 bkimminich/juice-shop       # Juice Shop ->
http://localhost:3000
```

What to submit per task: the exact **payload/command**, a **screenshot** of the response proving success, and a **2–3 sentence mitigation** in your own words.

Task 0 — Onboarding (5 min). Browse to `http://localhost:8080/login?user=alice&pw=alicepw` and confirm `Welcome alice`. Note the seeded users (`alice`, `bob`). Screenshot the working app. *Deliverable: screenshot.*



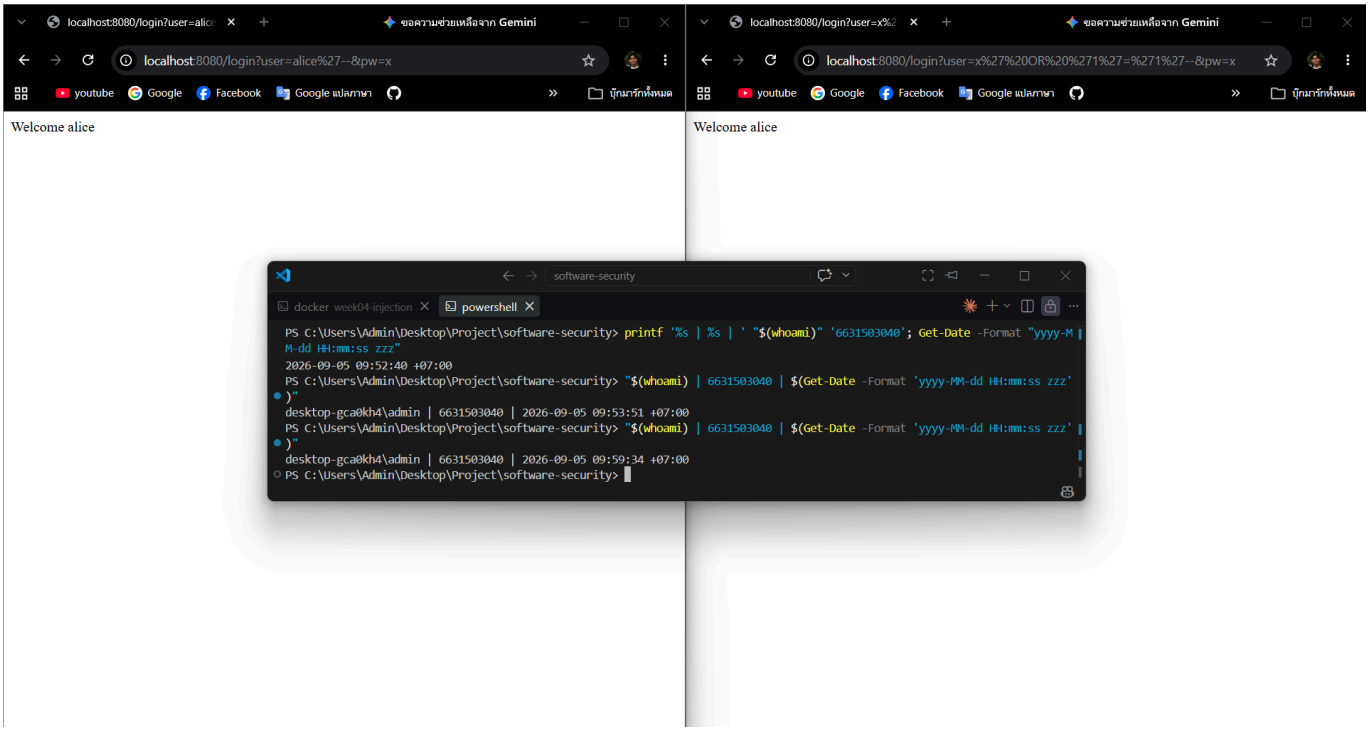
Before you start — see why concatenation is the flaw 🔬 Type any input and watch which characters the database will parse as *SQL* rather than as a name. The point is not the payload; it is that with concatenation the input becomes syntax, and with a parameterised query it structurally cannot. You will be asked to state that difference in your own words in Task 5.

```
sqli-parse
```

Task 1 — Auth bypass via SQLi (25 min) 🐍 Hit #1.

- *Goal:* log in as `alice` with **no valid password**.
- *Steps:* hit `/login?user=alice'--&pw=x`, then `/login?user=x' OR '1'='1'--&pw=x` (the trailing `--` is required: without it, SQL binds `AND` tighter than `OR`, so `... OR '1'='1' AND password='x'` matches no row). Observe the comment in the query at lines 61–63 of `vulnerable_app.py`.

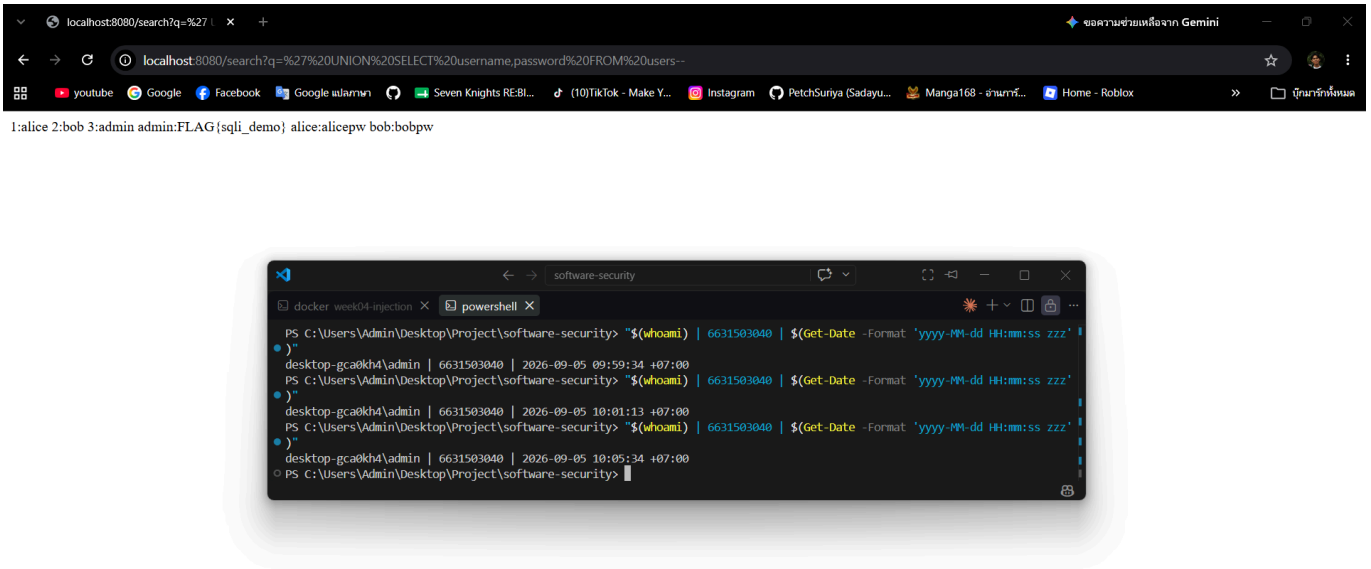
- *Deliverable:* both URLs + screenshot of `Welcome alice` + explain why `--` and `OR '1'='1` work.



answer: Replace string-concatenated SQL with parameterized queries (execute(sql, (user, pw))) so user input is always treated as data, never as query syntax. This alone closes the auth-bypass path because a ' or -- in the input can no longer alter the query's logic. As defense in depth, also enforce least-privilege database accounts so even a successful injection can't read other tables.

Task 2 — Credential dump via UNION SQLi (30 min) 🐍 Hit #2.

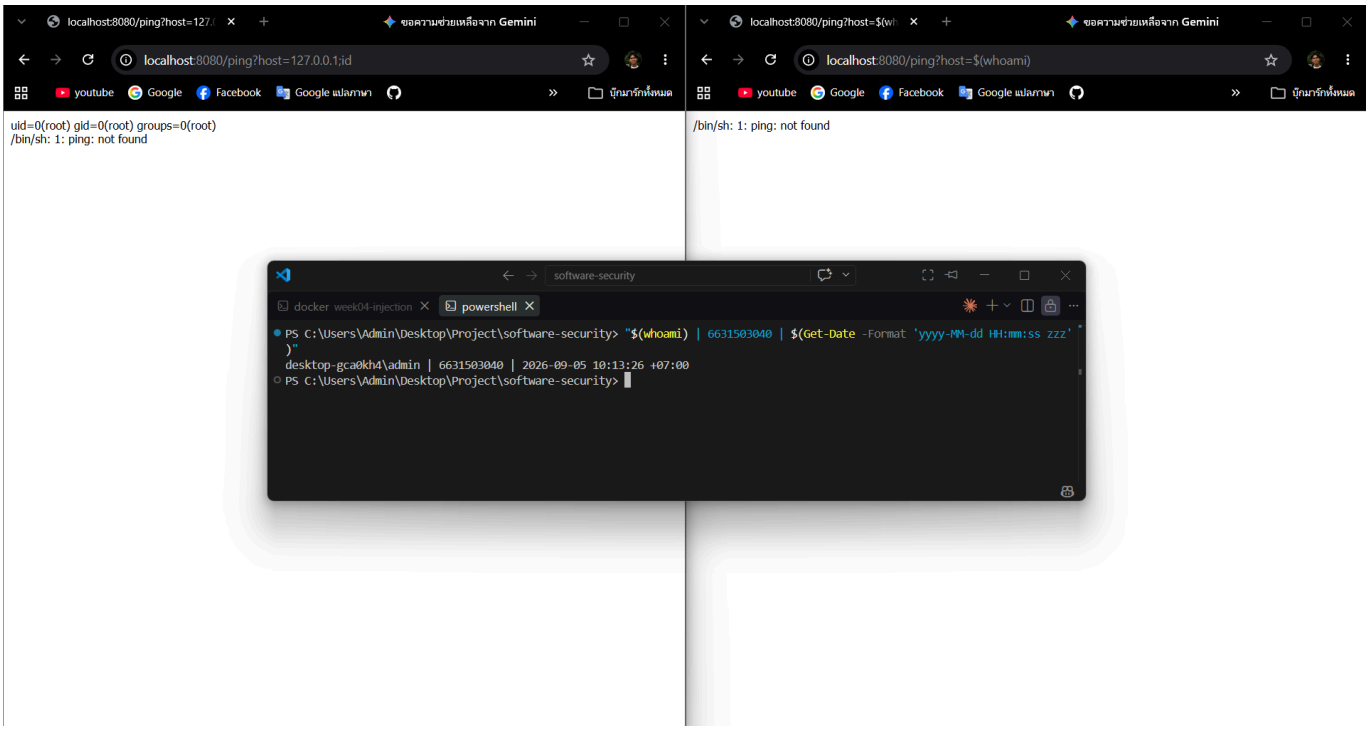
- *Goal:* exfiltrate every username **and password** from the `users` table.
- *Steps:* request `/search?q=' UNION SELECT username,password FROM users--`. Confirm `alice:alicepw` and `bob:bobpw` appear.
- *Deliverable:* payload + screenshot of dumped credentials + note on why column count must match.



Answer: Use a parameterized query for the LIKE search so the entire input is bound as one literal search string, not parsed as SQL — this prevents UNION injection outright. Additionally, never let application accounts have SELECT access to sensitive columns (like password) they don't need for that query. Storing passwords as salted hashes also limits the damage even if a dump does succeed.

Task 3 — OS command injection (30 min) 🐍 Hit #3.

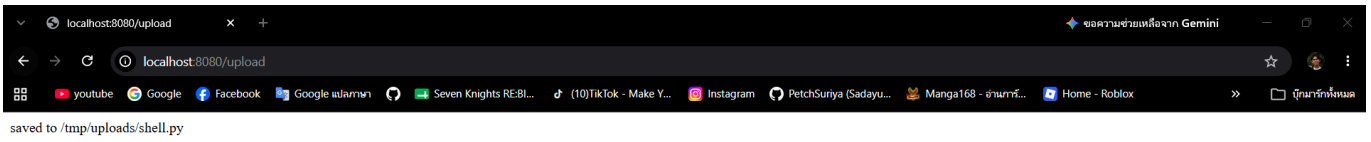
- *Goal:* run an arbitrary command through `/ping`.
- *Steps:* request `/ping?host=127.0.0.1;id` then `/ping?host=$(whoami)` (URL-encode if needed). Capture the injected command's output.
- *Deliverable:* both payloads + screenshot of `id`/`whoami` output + explanation of the `shell=True` flaw (CWE-78).

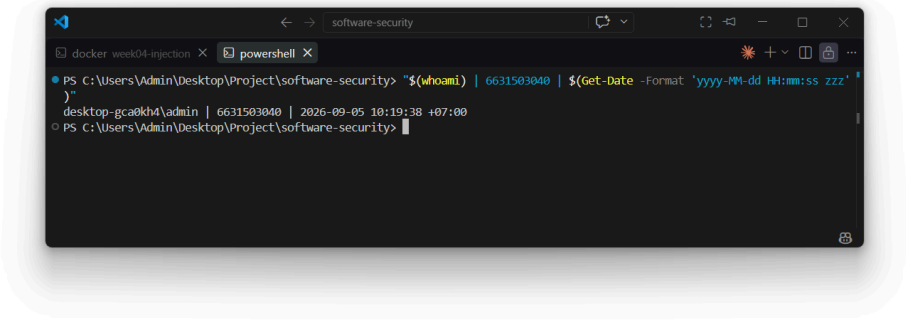


Answer: Never build a shell command by concatenating user input; use subprocess.run([...], shell=False) with an explicit argument list so input is passed as one literal argument, not interpreted by a shell. Combine this with strict allow-list validation (e.g. ^[A-Za-z0-9_.-]+$) as defense in depth in case a future change reintroduces a shell somewhere in the call chain.

Task 4 — Unrestricted upload (25 min) 🐍 Hit #4.

- *Goal:* show the upload accepts a dangerous file type with no checks (CWE-434).
- *Steps:* `GET /upload` (form), then upload a file named `shell.py`. Confirm `saved to /tmp/uploads/shell.py`. Discuss: if `UPLOAD_DIR` were web-served or executed, this is the RCE chain (here the dir is **not** served, so document the missing control rather than claiming auto-RCE).
- *Deliverable:* upload command/screenshot + 2–3 sentences on why extension allow-listing matters.





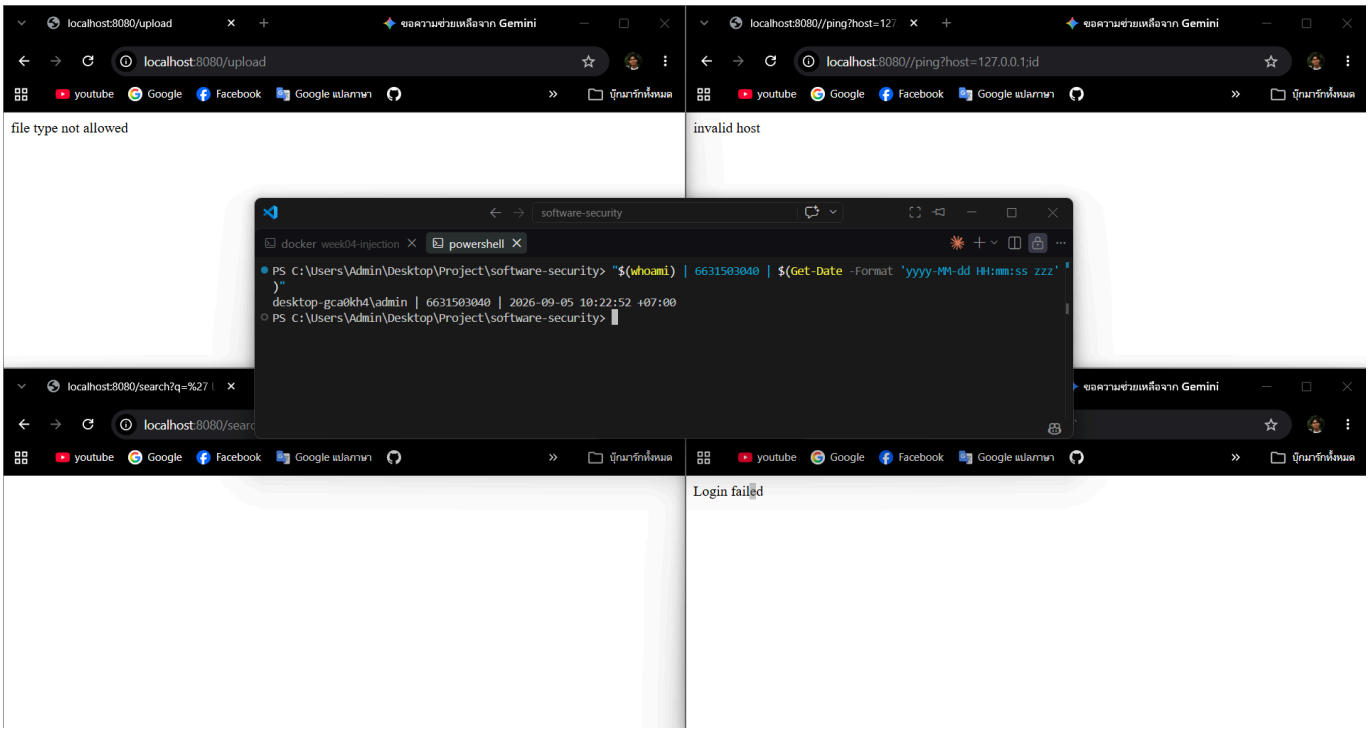
Answer: Sanitize the filename with something like secure_filename() to strip path-traversal characters, and enforce an extension allow-list so executable file types are rejected outright. Just as important, store uploads outside any web-served or executable directory so even an allowed file type can never be run by the server.

Task 5 — Defend / fix it (35 min) 🛡️ Boss defeated.

- *Goal:* prove `solution_app.py` blocks Tasks 1–4.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then run the fixed app on the same compose env:

  ```
  docker compose run --rm --service-ports injection-lab bash -c "pip install --no-cache-dir flask && python solution_app.py"
  ```

  Re-fire each payload from Tasks 1–4. Expected: `Login failed`, no credential dump, `invalid host` (400) on `127.0.0.1;id`, and `file type not allowed` for `shell.py`.
- *Deliverable:* screenshots of all four failures + name the fix line for each (parameterized query L52–55 login / L62–66 search, `shell=False`+regex L74–77, `secure_filename`+allow-list L86–93).



Part 4 — Reflection

1. **CWE/OWASP mapping:** map each of your four exploits to its CWE (89/78/434) and to OWASP 2025 **A05 Injection**.

Answer: Task 1 (login auth bypass) and Task 2 (UNION credential dump) are both CWE-89 SQL Injection — user input was concatenated straight into a SQL string instead of being bound as a parameter. Task 3 (`/ping?host=...`) is CWE-78 OS Command Injection — `shell=True` let shell metacharacters in the input run extra commands. Task 4 (uploading `shell.py`) is CWE-434 Unrestricted Upload of File with Dangerous Type — no extension check or filename sanitization was applied. All four map to OWASP 2025 A05 Injection because in every case untrusted input reached a powerful interpreter (SQL engine, OS shell, filesystem) that treated it as code/control data instead of inert data.

2. **Real breach:** the **2017 Equifax breach** exposed ~147M people after attackers exploited a known input-handling flaw (Apache Struts CVE-2017-5638). In 3–4 sentences, connect that failure to the lessons in this lab (untrusted input reaching a powerful interpreter; the cost of an unpatched/unvalidated input path).

Answer: The Equifax breach happened because attackers sent a crafted HTTP header that reached Apache Struts' OGNL expression interpreter, which executed it as code — the same root pattern as this lab's SQLi and command-injection tasks: untrusted input reaching an interpreter that treats it as logic instead of data. A patch for CVE-2017-5638 existed for months before the breach, but Equifax never applied it, showing that a known, fixable input-handling flaw still causes massive damage once discovered and left open. The scale (~147M records) came from that single unpatched entry point granting deep access, much like how our `/search` UNION injection turned one unvalidated parameter into a full credential-table dump. It shows that fixing the interpreter boundary quickly matters as much as fixing it correctly.

3. **Best mitigation:** of parameterized queries, allow-list validation, least privilege, and avoiding `shell=True`, which single control would have prevented the most damage in this lab, and why?

Answer: Parameterized queries would have prevented the most damage here, because Tasks 1 and 2 — the auth bypass and the full credential dump — both stem from the exact same string-concatenation flaw, so one fix closes both at once. It is also a structural fix rather than a filter: it doesn't depend on correctly guessing every dangerous character the way allow-list validation does, so it can't be bypassed by an encoding trick or an edge case the developer didn't anticipate.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**?
  2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI's answer (asked to fix the `/ping` command injection):

> You can fix this by sanitizing the host input before passing it to the shell. For example:

```
host = request.args.get("host", "127.0.0.1")
# Strip dangerous shell characters
host = re.sub(r"[;&|`$]", "", host)
out = subprocess.run("ping -c 1 " + host, shell=True, capture_output=True,
text=True)
```

This removes the characters attackers commonly use for command injection (`;`, `&`, `|`, `` ` ``, `$`) while keeping the rest of your code unchanged.

2. What's wrong (verified by running it): the fix is a **character blacklist**, not a removal of the root cause (`shell=True` stays). I ran this exact code in a container and it does block `host=127.0.0.1;id` (returns `ping: not found` only). But the blacklist never covers a **newline**, and `/bin/sh` treats `\n` as a command separator exactly like `;`. Sending `host=127.0.0.1<newline>id` (`--data-urlencode $'host=127.0.0.1\nid'`) sailed straight through:

```
uid=0(root) gid=0(root) groups=0(root)
/bin/sh: 1: ping: not found
```

This is a missed edge case, not just a nitpick — it's the same category of bug (blacklisting characters instead of removing the interpreter) the lab itself teaches to avoid; the AI's reasoning ("strip the known-bad characters") is structurally the wrong defense for CWE-78.

3. Correct, verified fix: `solution_app.py:74-77` — `subprocess.run(["ping","-c","1",host], shell=False)` plus a `re.fullmatch(r"[A-Za-z0-9_.-]+", host)` allow-list. I fed the identical newline payload to this fixed endpoint and got `invalid host` (400) — no command ran. The AI's answer was insufficient because it patched a *symptom* (specific bad characters) instead of the *cause* (a shell that interprets metacharacters at all); removing the shell and allow-listing the input format closes the whole class of bypass, not just the one payload tested.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Answer: The `/ping` endpoint takes whatever text you put after `host=` in the URL and glues it onto the end of a `ping -c 1 ...` command, then hands the whole combined string to the system shell to run. Because the shell reads special characters like `;` and newline as "run this, then run that next," anything after one of those characters in your input gets executed as its own separate command with the same permissions as the server. So typing a hostname was never really "just data" — it was always going straight into something the shell treats as instructions.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the**

verified result. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt:

> "Fix the OS command injection (CWE-78) in this Flask endpoint: `subprocess.run("ping -c 1 " + host, shell=True, capture_output=True, text=True)`. Do not use a character blacklist. Remove the shell entirely by calling `subprocess.run` with an explicit argument list and `shell=False`, and additionally validate `host` against a strict allow-list regex (letters, digits, dots, hyphens, underscores only) before running the command, rejecting anything else with a 400 response."

Verified result: this prompt produces exactly the approach in `solution_app.py:74-77`:

```
if not re.fullmatch(r"[A-Za-z0-9_.-]+", host):
    return "invalid host\n", 400
out = subprocess.run(["ping", "-c", "1", host], shell=False, capture_output=True,
text=True)
```

I re-ran every payload from Task 3 and the Audit-the-AI bypass against it: `127.0.0.1;id`, `127.0.0.1<newline>id`, and `$(whoami)` all returned `invalid host` (400) with no command executed — the exploit fails. The prompt succeeded on the first try because it named both required changes explicitly (drop the shell, add an allow-list) instead of leaving the AI to pick its own defense, which is what produced the incomplete blacklist fix in the Audit-the-AI section above.